Multi-Core Global Scheduling —

Milestone 3: Critical Evaluation and Transfer

Md Mostafizur Rahman
Electronic Engineering, Hochschule Hamm-Lippstadt (HSHL)
Lippstadt, Germany md-mostafizur.rahman@stud.hshl.de

***Abstract*—Critical evaluation of Gracioli et al. (2013) and a concrete transfer to a realistic, accessible context. The evaluation separates concrete strengths from concrete limitations, assesses scalability and applicability, and then reports my own transfer experiment (Windows/Ubuntu WCET measurement + UPPAAL verification) as the transfer step required by this milestone. Structured notes, not publication prose.**

I. Strengths (Concrete)

- **OS isolated as an independent variable.** The paper is the first apples-to-apples G-EDF/P-EDF comparison on a purpose-built RTOS against Linux RT. The resulting *reversal* finding — OS quality can outrank algorithm choice in light-utilisation, short-period regimes — is new and actionable, not a generic "it depends".
- **Zero-cost policy selection.** Compile-time metaprogramming resolves global/partitioned choice with no runtime branch, cleanly separating policy (Criterion) from mechanism (Queue). Strong software-engineering contribution and a direct code-reuse demonstration.
- **Principled overhead accounting.** Preemption-centric inflation folds all five measured sources into e'_i before the tests, so measurement and schedulability analysis are coupled rather than reported side by side.
- **Breadth of design space.** 18 task-set families, tests with/without overhead, up to 100 processors ($\sim$1.1 CPU-years), plus CPMD via hardware performance counters — a wide, reproducible sweep.
- **Sound compression of results.** The utilisation-weighted schedulability metric $W(D_c)$ reduces CPMD-dependent surfaces to interpretable 2-D curves without hiding the load dependence.

II. Limitations, Risks, and Hidden Assumptions (Concrete)

- **Sampled bounds for HRT claims.** Overhead is "observed worst case" over many runs, not analytically bounded. A single unsampled tail breaks the guarantee. This is compounded by *non-standard outlier removal*: 2 ms context-switch spikes are discarded, yet for HRT those spikes are exactly the deadline misses.
- **Ratios measure test pessimism as much as feasibility.** All tests are sufficient-only; the light-bimodal G-EDF ratio collapses to 0 at cap $\approx$45 (100 proc), wasting $\approx$55% of capacity to test conservatism. Cross-algorithm "schedulability" therefore partly compares *test maturity*, not just the algorithms.
- **Workload understates interference.** A CPU-bound loop with a tiny working set minimises CPMD and memory contention. The paper's own CPMD-vs-WSS data show G-EDF and P-EDF converge for WSS $\geq$512 KB — real vision/ML/sensor-fusion workloads would erode G-EDF's advantage that the synthetic workload preserves.
- **Scalability claims are partly theoretical.** Overhead is measured only on 8 logical cores; the 100-processor results are overhead-free. Large-m conclusions rest on idealized tests.
- **No isolation, no certification.** Library mode shares an address space $\Rightarrow$ no fault isolation, no ASIL-D/DO-178C path. Results are a valid research baseline, not a deployable stack.
- **Platform specificity.** x86 Sandy Bridge with QPI; ARM/other interconnects would change every measured value.
- **Omitted winning paradigm.** Clustered EDF (C-EDF-L2) already beat both G-EDF and P-EDF on Linux (Bastoni). Restricting to a two-way comparison answers a narrower question than the state of the art.

III. Scalability Assessment

- **Runtime.** G-EDF's $O(n)$ ordered list is the visible bottleneck above $\approx$75 threads; a heap would give $O(\log n)$. P-EDF's $O(n/m)$ scales with cores.
- **Memory / contention.** Global queue + single lock concentrates contention; partitioned queues spread it but fragment spare capacity.
- **System size.** No measured overhead beyond 8 logical cores $\Rightarrow$ conclusions for tens/hundreds of cores are extrapolations.

IV. Applicability to Realistic Embedded Scenarios

Suitable:

- Deeply embedded, no-MMU nodes with static, light-utilisation, short-period workloads (IoT / high-frequency control): the overhead-to-period ratio dominates, so EPOS library-mode G-EDF's low overhead is decisive.
- RTOS design and overhead baselining / research prototyping.

Unsuitable / problematic:

- Safety-certified automotive/avionics: no isolation, sampled (not analytic) bounds, no certification artefacts.

TABLE I
SAMPLED WCET, SAME HARDWARE

OS	No load	Loaded	Increase	Relative
Windows 11	437 ms	819 ms	382 ms	87.3%
Ubuntu Linux	309 ms	458 ms	149 ms	48.5%
Ubuntu (plain run)	301 ms	478 ms	—	consistent

TABLE II
LINUX PERF COUNTERS (FULL PROGRAM)

Counter	Value
Context switches	18,220
CPU migrations	13
Cache references	3.484×10^9
Cache misses (rate)	7.135×10^6 (0.20%)
Cycles	3.761×10^{11}
Instructions (IPC)	1.014×10^{12} (2.70)

- Cache-heavy, large-WSS workloads: migration CPMD erodes the utilisation advantage; clustered scheduling is the better fit.
- Mixed-criticality, sporadic, or constrained-deadline ($d_i < p_i$) systems: outside the implicit-deadline periodic model.
- Large-core-count deployments needing *measured* rather than idealized overhead.

V. TRANSFER: MY EXPERIMENTAL STUDY

Objective. Test whether the paper's core claim — the OS substrate materially changes effective WCET / schedulability — (i) reproduces *qualitatively* on accessible commodity hardware and (ii) can be turned into a *formally verified* deadline consequence.

A. Setup

- Platform: Lenovo LOQ 15ARP9, AMD Ryzen 7 7435HS (8 physical / 16 logical, SMT), 16 MB shared L3; *same* hardware under Windows 11 and Ubuntu Linux.
- Target: integer loop $x \leftarrow x + i^2$, 5×10^6 iterations, small working set. Phase 1: 10 timed runs, no load. Phase 2: 10 timed runs under 15 background worker processes (separate processes to avoid the Python GIL). Sampled WCET = max observation (empirical, *not* an analytical HRT bound).
- Linux: `perf stat` counts context switches, CPU migrations, cache refs/misses, cycles, instructions over the whole process group (interference indicators, not isolated c^{xs}, c^{scd}, c^{pd}).

B. Results

- The same workload on the same silicon inflates WCET differently per OS and per load — a qualitative analogue of the EPOS–LITMUS^{RT} result.
- Low miss rate (0.20%) and high IPC (2.70) confirm the task stayed compute-bound; only 13 migrations mean the scheduler preserved placement. The slowdown is aggregate WCET inflation from CPU sharing + scheduling + context switching + SMT contention + cache effects — *not* a measurement of one named RTOS overhead source.

TABLE III
SELECTED UPPAAL VERDICTS (COMBINED MODEL)

Query	Verdict
A[] ¬deadlock	true
E◇ Safe.DoneAll	true
E◇ Miss.Missed	true
A[] ¬miss_deadline_miss	**false**

C. Formal verification (UPPAAL)

- Two-core G-EDF model, integer-millisecond clocks, two independent automata (safe, miss). Both jobs released at $t = 0$; two free cores dispatch immediately without preemption. Measured $T_1 = 309$, $T_2 = 458$ ms.
- Demonstration deadline $= 400$ ms. The loaded location's invariant uses the deadline as the tighter bound, so the completion edge guarded by $y_2 = 458$ is structurally unreachable — the formal expression of "the task cannot finish in time". Terminal locations carry self-loops to avoid artificial deadlock.
- The falsified safety property yields the counterexample: a 458 ms effective WCET *cannot* meet a 400 ms deadline on any path, even with two cores and immediate G-EDF dispatch. Adding processing resources does not remove a per-job timing violation.

D. What the transfer does and does not show

- **Shows:** a coherent evidence chain — the reference identifies WCET-inflation sources; my benchmark measures aggregate inflation on real hardware; Linux counters reveal the scheduling/cache activity behind it; UPPAAL proves the deadline consequence of the measured values.
- **Does not show (honest scope):** Windows/Ubuntu are neither EPOS nor LITMUS^{RT}; a max of 10 runs is a sampled WCET that may miss rare events; `perf` aggregates the process group and cannot separate $c^{xs}, c^{tck}, c^{ipi}, c^{pd}$; the UPPAAL model abstracts away arbitrary releases, migration, and IPI.
- **Model-consistency caveat.** A utilisation-only view misleads under $d_i < p_i$: two loaded Linux tasks at period 1000 ms give $U = 2(458/1000) = 0.916 < 1$, which an implicit-deadline test would accept — yet a 400 ms deadline is missed by a 458 ms job. Constrained deadlines require deadline-aware response-time analysis, not $U \leq 1$.

VI. EXTENSIONS, ADAPTATIONS, AND OPEN QUESTIONS

1) **Heap in EPOS.** Replace the $O(n)$ list with a heap ($O(\log n)$) and re-measure the crossover — does G-EDF(EPOS) beat P-EDF(LITMUS^{RT}) into *medium* utilisation, not only light?
2) **Clustered/semi-partitioned on EPOS.** Implement C-EDF-L2 on EPOS and test whether the RTOS advantage

amplifies clustered's known win over both G-EDF and P-EDF.

3) **Measurement rigor.** Move from aggregate `perf` to per-source isolation: kernel tracing, pinned affinity, RT priority, explicit warm-up, high-percentile/max over many repetitions — ideally on a real RTOS testbed.
4) **Richer formal model.** Extend the UPPAAL model with sporadic arrivals, constrained deadlines, resource blocking, and explicit migration/IPI; generalise from a two-job demo to a reusable multicore-EDF timed-automata template.
5) **ARM port.** Repeat on ARM Cortex-A (different coherence, IPI, in-order pipeline) to test how far the x86 numbers transfer.
6) **Certification bridge.** What would an isolation-preserving, analytically-bounded EPOS variant need to be viable for ASIL-D — and how much overhead would isolation cost?